

Q-Bar Challenge (Barrinha Cup) – Implementação de Linear Q-Learning em Haskell

Objetivo: Desenvolver um agente autônomo baseado em Aprendizado por Reforço (Reinforcement Learning), utilizando o algoritmo Linear Q-Learning na linguagem puramente funcional Haskell. O projeto visa aplicar a Mônada de Estado (StateT) e a biblioteca hmatrix para treinar um agente capaz de controlar uma barra (raquete) em um ambiente dinâmico, maximizando sua recompensa ao longo do tempo.

Introdução

O Aprendizado por Reforço (*Reinforcement Learning* - RL) é uma área do aprendizado de máquina onde um agente aprende a tomar decisões interagindo com um ambiente. O objetivo é descobrir uma política de ações que maximize um sinal de recompensa a longo prazo, processo este formalizado através de Processos de Decisão de Markov (MDPs).

O algoritmo **Q-Learning** é um método clássico de controle *off-policy* baseado em Diferença Temporal. Para ambientes com espaços de estados contínuos ou muito vastos (como posições e velocidades da bola e da barra na nossa simulação), o Q-Learning tabular torna-se inviável. Para contornar a "maldição da dimensionalidade", o projeto utiliza a **Aproximação Linear de Função** (Linear Q-Learning), onde a função de valor de ação é aproximada pela seguinte combinação linear:

$$Q(s, a) \approx \theta^T \phi(s, a)$$

Onde $\phi(s, a)$ é o vetor de características (*features*) do ambiente e da ação, e θ é o vetor de pesos que o agente ajusta durante o treinamento.

O grande diferencial deste projeto é a implementação no paradigma puramente funcional usando **Haskell**. Para gerenciar a mutabilidade inerente ao treinamento de um agente de RL sem ferir a pureza da linguagem, a arquitetura baseia-se no transformador de mônada StateT, isolando a física do jogo e a inteligência do agente, enquanto a biblioteca hmatrix lida de forma otimizada com as operações de álgebra linear.

Descrição das Atividades

O projeto está sendo desenvolvido e organizado cronologicamente nas seguintes etapas:

- Definição da Arquitetura e Modelagem do MDP (Realizado - 1º Commit):**
 - Estruturação dos tipos de dados: Campo (física da bola e da barra) e CraqueState (vetor de pesos e taxa de exploração ϵ).
 - Configuração do transformador de Mônada StateT Mundo IO para encapsular o estado global sem variáveis globais impuras.
- Engenharia de Características (Feature Extraction):**

- Implementação da função `extrairFeatures`, traduzindo o estado bruto (distância da bola, posição) em um vetor numérico manipulável pela biblioteca `hmatrix`.
- 3. **Implementação da Política de Exploração:**
 - Construção da lógica ϵ -greedy usando `System.Random`, balanceando a exploração de novas jogadas e a exploração (ações gulosas baseadas no produto escalar $\phi < \theta$).
- 4. **Mecânica de Física e Recompensa (O "Juiz"):**
 - Programação do motor físico do jogo para atualizar posições, rebater a bola e atribuir pontuações (recompensas) ou penalidades (deixar a bola cair).
- 5. **Atualização de Pesos (O "Treinador"):**
 - Implementação do cálculo de gradiente descendente estocástico para atualizar o vetor de pesos θ após cada iteração.
- 6. **Documentação e Entrega Final:**
 - Elaboração das instruções de execução, empacotamento do código-fonte e detalhamento final da divisão de tarefas (ex: Membro A e B na Lógica Funcional/Monads, Membro C na Física/Ambiente, Membro D na Álgebra Linear/Q-Learning).

Resultados

(Parciais). Até o momento, o primeiro commit foi realizado com sucesso, estabelecendo a espinha dorsal do projeto. O esqueleto da Mônada de Estado (`StateT`) provou-se eficaz para separar o domínio da física (`Campo`) da inteligência artificial (`CraqueState`). O uso do operador de produto escalar da `hmatrix` já está integrado à função de estimativa do valor Q , validando a escolha das bibliotecas. *(Na versão final, esta seção conterá os gráficos de convergência de recompensa e o desempenho do agente na "Barrinha Cup").*

Discussão

(Prévia). A implementação de algoritmos de Reinforcement Learning em linguagens puramente funcionais traz desafios únicos de *design*. Em linguagens imperativas, a atualização dos pesos θ seria uma simples reatribuição de variáveis. Em Haskell, a imposição da imutabilidade nos forçou a desenhar uma arquitetura mais robusta e segura usando `StateT`. Observa-se, nesta fase inicial, que essa obrigatoriedade torna o código muito mais previsível e modular, permitindo que a função de atualização de pesos opere como uma transformação pura de estados. Resta avaliar, nas próximas etapas, o impacto computacional dessa estrutura durante o loop de treinamento intensivo.

Conclusão

(Prévia). O planejamento estrutural demonstra que a combinação de Haskell com Linear Q-Learning é não apenas viável, mas promove um design de software altamente rigoroso. A base estabelecida pelo uso de `hmatrix` e `StateT` garante que o ambiente "Barrinha Cup" (Q-Bar

Challenge) possa ser simulado de forma eficiente. O foco das próximas etapas será o refinamento matemático do algoritmo de atualização e o ajuste dos hiperparâmetros para garantir a convergência do agente autônomo.

Referências

- Ruiz, A. (2005). *Numeric Linear Algebra in Haskell (hmatrix)*.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
- Wadler, P. (1995). *Monads for functional programming*. In *Advanced Functional Programming*. Springer.
- Watkins, C. J. C. H., & Dayan, P. (1992). *Q-learning*. *Machine Learning*, 8(3-4), 279-292.

Palavras-Chave

Frente 1: Teoria de Reinforcement Learning

- *Linear Function Approximation Q-Learning*
- *Feature extraction reinforcement learning*
- *Gradient descent for linear Q-Learning*
- *Epsilon-greedy exploration strategy*

Frente 2: Implementação em Haskell

- *Haskell hmatrix tutorial* (A biblioteca `hmatrix` é excelente para lidar com vetores, matrizes e produtos escalares de forma eficiente).
- *State Monad tutorial Haskell* (Essencial para não ter que passar o vetor de pesos e o estado do ambiente manualmente em cada assinatura de função).
- *Purely functional reinforcement learning*
- *Tail recursion state update Haskell*